

CO1-AT3 - CODING CHALLENGE

[View Only](#)
[← Back](#)

QUESTIONS

Q1

 Smart University Payroll
Management System
25 marks

Q2

 Smart Vehicle Insurance Claim
System
25 marks


2/2 answered

Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty

101

Dr. Arun

CSE

50000

20

Your Code

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Employee[] emp = new Employee[3];
7
8         emp[0] = new TeachingFaculty(101, "Dr. Arun", "CSE", 50000, 20);
9         emp[1] = new TechnicalStaff(102, "Mr. Ravi", "ECE", 35000, 15);
10        emp[2] = new VisitingFaculty(103, "Dr. Kumar", "IT", 40, 1200);
11
12        double total = 0;
13        Employee highest = emp[0];
14
15        System.out.println("Employee Details\n");
16
17        for (Employee e : emp) {
18            e.displayEmployee();
19            total += e.calculateSalary();
20            if (e.calculateSalary() > highest.calculateSalary())
21                highest = e;
22        }
23
24        System.out.println("Highest Paid Employee");
25        System.out.println("Employee ID : " + highest.employeeId);
26        System.out.println("Salary : " + highest.calculateSalary());
27    }
28 }
```

Input

```
3
Teaching Faculty
101
Dr.Arun
---
```

Output

```
Employee Details
Employee ID : 101
Name : Dr. Arun
Department : CSE
Monthly Salary : 60000.0
```

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

0.00 / 25 marks

```
27
28     System.out.println("\nAverage Salary");
29     System.out.printf("%.2f\n", total / emp.length);
30
31     int searchId = 102;
32     System.out.println("\nEmployee Found");
33     for (Employee e : emp) {
34         if (e.employeeId == searchId) {
35             System.out.println("Employee ID : " + e.employeeId);
36             System.out.println("Name : " + e.employeeName);
37             System.out.println("Salary : " + e.calculateSalary());
38         }
39     }
40 }
41 }
42
43 class Employee {
44     int employeeId;
45     String employeeName;
46     String department;
47     double basicSalary;
48
49     Employee() {}
50
51     Employee(int id, String name, String dept, double salary) {
52         employeeId = id;
53         employeeName = name;
54         department = dept;
55         basicSalary = salary;
```

```
59     return basicSalary;
60 }
61
62 void displayEmployee() {
63     System.out.println("Employee ID : " + employeeId);
64     System.out.println("Name : " + employeeName);
65     System.out.println("Department : " + department);
66     System.out.println("Monthly Salary : " + calculateSalary());
67     System.out.println();
68 }
69 }
70
71 class TeachingFaculty extends Employee {
72     int teachingHours;
73     double incentivePerHour;
74
75     TeachingFaculty(int id, String name, String dept, double salary, i
76         super(id, name, dept, salary);
77         teachingHours = hours;
78         incentivePerHour = incentive;
79     }
80
81     double calculateSalary() {
82         return basicSalary + (teachingHours * incentivePerHour);
83     }
84 }
85
86 class TechnicalStaff extends Employee {
87     int overtimeHours;
```



```
92     overtimeHours = hours;  
93     overtimeRate = rate;  
94 }  
95  
96 double calculateSalary() {  
97     return basicSalary + (overtimeHours * overtimeRate);  
98 }  
99 }  
100  
101 class VisitingFaculty extends Employee {  
102     int lecturesHandled;  
103     double paymentPerLecture;  
104  
105     VisitingFaculty(int id, String name, String dept, int lectures, double payment) {  
106         super(id, name, dept, 0);  
107         lecturesHandled = lectures;  
108         paymentPerLecture = payment;  
109     }  
110  
111     double calculateSalary() {  
112         return lecturesHandled * paymentPerLecture;  
113     }  
114 }
```



TN10AB1234

Ramesh

800000

Bike

TN09XY5678

Suresh

100000

Truck

TN11TR9999

Logan

1500000

Search Vehicle:

TN09XY5678

Expected Output

Vehicle : TN10AB1234

Claim Amount : 640000.0

Vehicle : TN09XY5678

Claim Amount : 60000.0

Vehicle : TN11TR9999

Claim Amount : 1350000.0

Highest Claim Vehicle

TN11TR9999

CO1-AT3 - CODING CHALLENGE

[View Only](#)
[← Back](#)

QUESTIONS

Q1

 Smart University Payroll
Management System
25 marks

Q2

 Smart Vehicle Insurance Claim
System
25 marks


2/2 answered

Q2 Smart Vehicle Insurance Claim System

25 marks

Problem Statement An insurance company processes claims for three categories of vehicles: Car, Bike, and Truck. The claim amount depends on the vehicle type. **Requirements** Superclass: Vehicle Private members: vehicleNumber ownerName insuredAmount Subclasses: Car Bike Truck Each subclass overrides calculateClaimAmount() using different formulas. The application should: Read details for N vehicles Store them using an array of superclass references Calculate claim amounts using runtime polymorphism Search by vehicle number Display the vehicle with the highest claim amount **Question 3 (Very Hard) Online Shopping Order Processing System (50 Marks)** **Problem Statement** An e-commerce company sells products in three categories: Electronics Clothing Grocery The billing process varies for each category. **Requirements** Superclass: Product Private members: productId productName price quantity Subclasses: Electronics Clothing Grocery Each subclass overrides calculateBill(). The program should: Store products in an array of superclass references Calculate the final bill using runtime polymorphism Search products by ID Display the most expensive order Compute total revenue

Claim Formula (Recommended)

Car

Claim = insuredAmount × 0.80

Bike

Claim = insuredAmount × 0.60

Truck

Your Code

```

1 import java.util.*;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Scanner sc = new Scanner(System.in);
8
9         int n = Integer.parseInt(sc.nextLine());
10
11         Vehicle[] vehicles = new Vehicle[n];
12
13         for (int i = 0; i < n; i++) {
14
15             String type = sc.nextLine().trim();
16             String vehicleNumber = sc.nextLine().trim();
17             String ownerName = sc.nextLine().trim();
18             double insuredAmount = Double.parseDouble(sc.nextLine().trim());
19
20             if (type.equalsIgnoreCase("Car")) {
21                 vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
22             } else if (type.equalsIgnoreCase("Bike")) {
23                 vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
24             } else if (type.equalsIgnoreCase("Truck")) {
25                 vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
26             }
27         }
28     }
29 }

```

Input

```

3
Car
TN10AB1234
Ramsh
-----

```

Output

```

Vehicle : TN10AB1234
Claim Amount : 640000.0
Vehicle : TN09XY5678
Claim Amount : 60000.0
Vehicle : TN11TR9999
Claim Amount : 1350000.0

```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



0.00 / 25 marks

```
29 String searchVehicle = sc.nextLine().trim();
30
31 Vehicle highest = vehicles[0];
32
33 for (Vehicle v : vehicles) {
34     System.out.println("Vehicle : " + v.getVehicleNumber());
35     System.out.println("Claim Amount : " + v.calculateClaimAmount());
36
37     if (v.calculateClaimAmount() > highest.calculateClaimAmount())
38         highest = v;
39 }
40
41
42 System.out.println("Highest Claim Vehicle");
43 System.out.println(highest.getVehicleNumber());
44 System.out.println(highest.calculateClaimAmount());
45
46 boolean found = false;
47
48 for (Vehicle v : vehicles) {
49     if (v.getVehicleNumber().equalsIgnoreCase(searchVehicle))
50         System.out.println("Vehicle Found");
51         System.out.println("Owner : " + v.getOwnerName());
52         System.out.println("Claim Amount : " + v.calculateClaimAmount());
53         found = true;
54         break;
55 }
56 }
57
```

```
77
78     public String getVehicleNumber() {
79         return vehicleNumber;
80     }
81
82     public String getOwnerName() {
83         return ownerName;
84     }
85
86     public double getInsuredAmount() {
87         return insuredAmount;
88     }
89
90     public double calculateClaimAmount() {
91         return 0.0;
92     }
93 }
94
95 class Car extends Vehicle {
96
97     public Car(String vehicleNumber, String ownerName, double insuredAmount) {
98         super(vehicleNumber, ownerName, insuredAmount);
99     }
100
101     @Override
102     public double calculateClaimAmount() {
103         return getInsuredAmount() * 0.80;
104     }
105 }
```